Function (User-Defined Function - UDF)

- **Purpose: A function is designed to perform a specific task and return a single value (like performing a calculation).**
- **Returns: It always returns a single value (scalar), which can be of any data type (e.g., `INT`, `VARCHAR`).**
- **Usage: functions can be used in SQL queries like:**
    1. **`SELECT` (e.g., for computations or retrieving values)**
    2. **`WHERE` (e.g., for filtering results)**

    However, Functions cannot be used to directly perform data-modifying operations such as: `INSERT` or `UPDATE` to alter or insert records into tables.

- **Parameters: Functions only accept `IN` parameters, which means you can only pass values to the function.**

You call a function in SQL like:

```
SELECT my_function(arg1);
```

-
- **Side Effects: Functions are generally not used for modifying data (such as inserting or updating records).**

Example:

```
DELIMITER //

CREATE FUNCTION add_numbers(a INT, b INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN a + b;
END //
DELIMITER ;

#CALLING A FUNCTION
SELECT add_numbers(2,3);
```

---

2. Stored Procedure

- **Purpose: A stored procedure is a set of SQL statements designed to perform tasks like data manipulation (e.g., `INSERT`, `UPDATE`, `DELETE`) and business logic.**
- **Returns: It can return multiple results or no result at all (depending on its logic). Unlike functions, stored procedures can return multiple values using `OUT` parameters or result sets.**
- **Usage: Procedures are used for executing a series of SQL statements in bulk. They are not typically used in `SELECT` or other SQL statements.**
- **Parameters: Stored procedures can accept `IN`, `OUT`, and `INOUT` parameters.**

Invoked by: You call a procedure using:
sql
Copy code

```
CALL my_procedure(arg1, arg2);
```

-
- **Side Effects: Stored procedures can modify data in the database (e.g., inserting or updating rows).**

Example:

```
SELECT * FROM DEP;

DELIMITER //
CREATE PROCEDURE add_customer(IN name VARCHAR(100))
BEGIN
   INSERT INTO DEP (FIRST_NAME) VALUES (name);
END //
DELIMITER ;

CALL add_customer("AMMU");
```

---

3. Trigger

- **Purpose: A trigger is an automatic action that gets executed when certain events (like `INSERT`, `UPDATE`, `DELETE`) happen on a specific table. It enforces rules, performs auditing, or maintains data integrity.**
- **Returns: Triggers do not return values. Their purpose is to execute automatically when an event occurs.**

- **Usage: Triggers are used to automatically respond to data changes Parameters: Triggers do not accept parameters; instead, they use special variables like `NEW` and `OLD` to reference the data before and after the event.**
- **Invoked by: Triggers are automatically executed when the specified event occurs on a table (e.g., after inserting a row).**
- **Side Effects: Triggers can modify data in the same or another table as part of their action.**

Example:

```
delimiter //
create trigger pos
before insert on dep
for each row
begin
set new.rating=abs(new.rating);
end//
delimiter ;

insert into dep(rating) values (-3);
select * from dep;
```

Key Differences:

Feature	Function	Stored Procedure	Trigger
Returns	Single value (scalar)	Multiple values, result sets, or none	No return values
Usage	In SQL statements (`SELECT`, `WHERE`)	Not used inside SQL statements; executed by `CALL`	Automatically executes when a table event occurs
Parameters	Only `IN`	`IN`, `OUT`, `INOUT`	No parameters
Modification of Data	Not used for data modification	Can modify data (e.g., `INSERT`, `UPDATE`)	Can modify data based on events (e.g., audit logs)
Execution	Called manually	Called manually	Executed automatically when event occurs

Side Effects	**No side effects**	**Can have side effects (data changes)**	**Can have side effects (data changes)**
Invocation	**Select statement**	**Called using `CALL`**	**Automatically invoked**

Q1. What are the main differences between a stored procedure, a function, and a trigger in MySQL?

- **Stored Procedure: A set of SQL statements that can return multiple results and perform complex operations.**
- **Function (UDF): A reusable function that returns a single value and can be used within SQL queries.**
- **Trigger: Automatically executes SQL in response to a specific event on a table.**

Q2. What is normalization, and why is it important?

In simple terms, normalization is the process of organizing data in a database to avoid duplication and redundancy. It involves dividing large tables into smaller ones and ensuring that relationships between them are properly maintained. This improves data consistency and efficiency.

Q3. When would you consider denormalization in a database?

Denormalization is MUCH NOT SIMILAR process of normalization. It involves combining smaller tables into larger ones, even if it leads to some redundancy, in order to improve read performance. This can make retrieving data faster but might increase storage needs and the risk of data inconsistency.

Q4. What is the difference between INNER JOIN and LEFT JOIN?
A1. INNER JOIN returns rows that have matching values in both tables, while LEFT JOIN returns all rows from the left table and matching rows from the right table. If no match, NULL is returned for columns from the right table.

Q5. How does a RIGHT JOIN work in MySQL?
A2. RIGHT JOIN returns all rows from the right table and the matching rows from the left table. If no match is found, NULL is returned for the left table columns.

Q6: -- correct it?
DELIMITER //
CREATE TRIGGER CAP()
BEFORE INSERT ON DEP

```
BEGIN
UPDATE NEW.FIRST_NAME= UPPER(FIRST_NAME) ;
END;
DELIMITER //

INSERT INTO DEP(FIRST_NAME) VALUES("ammu");
```